

A LECTURE FOR COMPUTER SCIENCE STUDENTS

Linear Algebra in Computer Science

Designed by

Dr. M. Asad Iqbal

How matrices, vectors, and dot products quietly power images, recommendation engines, and every neural network you'll ever train.

Data • Vectors • Matrices • Classifiers • Neural Networks

Why Does Computer Science Need Linear Algebra?

Linear Algebra is the backbone of computer science — especially AI and Machine Learning. It is the tool that lets a computer process, analyze, or compress numbers at massive scale.

1

Representation of Data

Images, text, and audio are stored as matrices — grids of numbers a computer can understand.

2

Dimensionality Reduction

Techniques like PCA and SVD shrink huge feature sets while keeping the important information.

3

Optimization

ML models use linear algebra to find the minimum error — the core engine of machine learning.

4

Feature Engineering

Linear transformations combine existing data into new, more meaningful features.

5

Similarity Measures

Embeddings turn words and sentences into vectors so machines can compare meaning.

Representation of Data

Complex datasets — images, text, audio — are stored in matrices (grids of numbers). That's how a computer “learns” and understands them.

No data or image is represented as a single line. Instead, matrices organize data into rows and columns, so each value knows its neighbours — essential when a filter needs nearby pixels to detect an edge.

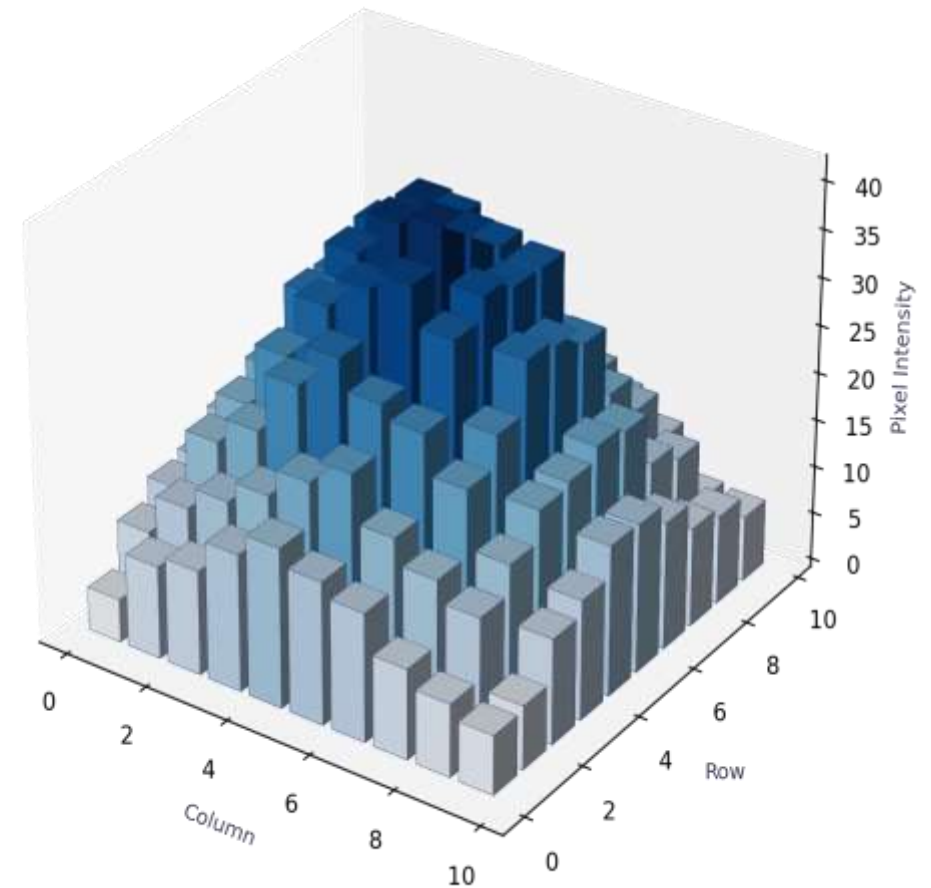
EXAMPLES

A 100×100 pixel image → becomes a 100×100 matrix

A WhatsApp photo → is internally a matrix

Netflix recommendations → matrix of user ratings

An Image Is Really a Matrix of Numbers



Dimensionality Reduction

Too many variables slow models down and cause two big problems: duplication of information, and models that can't find a clear pattern. Linear Algebra (SVD, PCA) reduces variables while keeping the important information — boosting model efficiency.

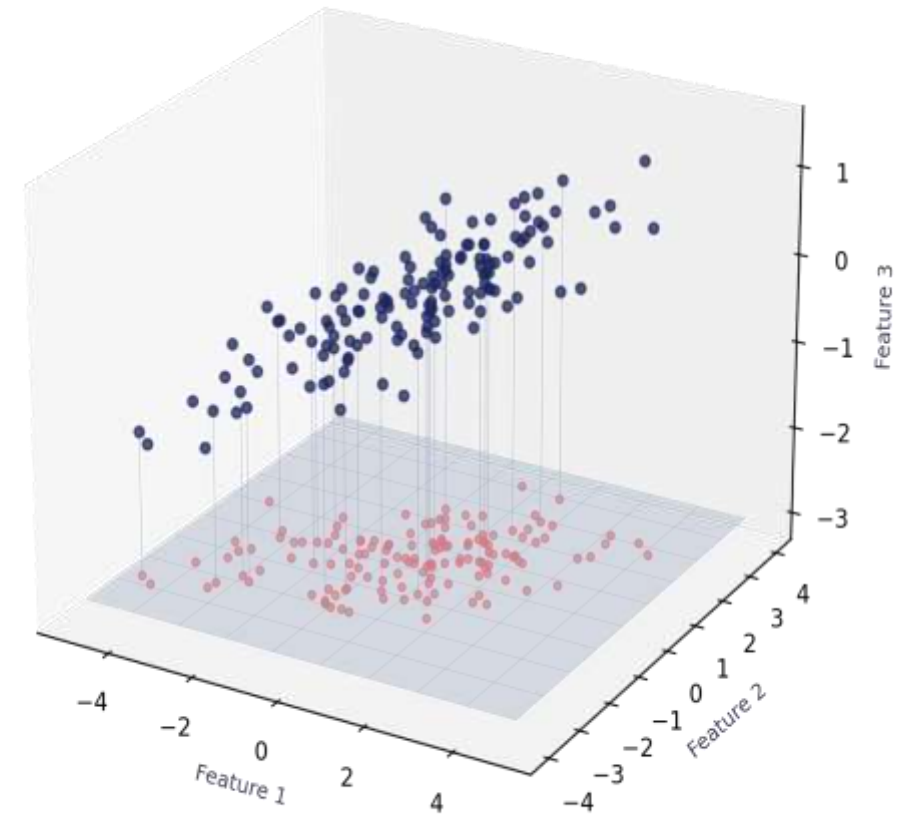
1,000 features → **reduced to 10** → model becomes faster

Used in PCA (Principal Component Analysis) for ML: less variation, easier to solve; more information, more useful.

WHY IT MATTERS

More dimensions → more duplication and no clear pattern. Fewer, well-chosen dimensions → the model recognizes patterns easily and generalizes better.

Dimensionality Reduction: 3D Data -> 2D Projection



Optimization — The Core Engine of ML

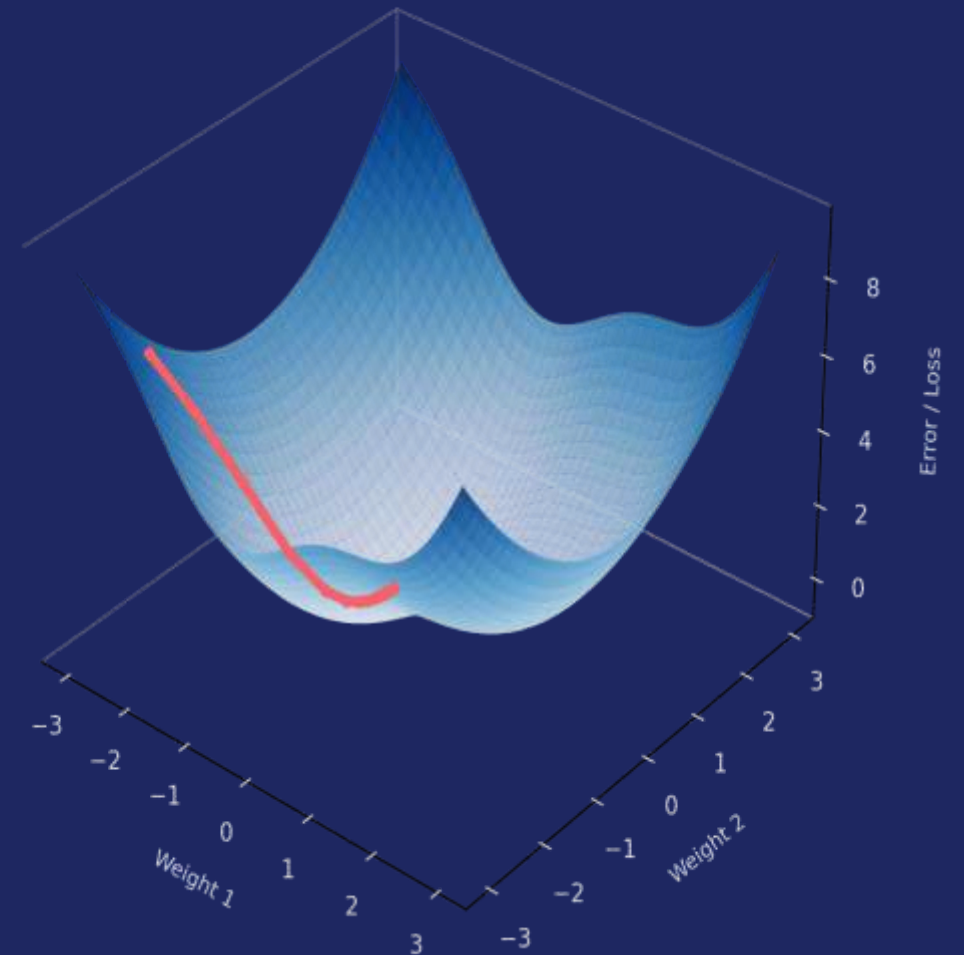
ML models use linear algebra to find the minimum of a function — the lowest possible error. Minimizing cost/error or maximizing accuracy/marks are the same idea: find the best option.

How training works:

1. Model starts with random weights (millions of numbers)
2. It makes a prediction and measures the error
3. Gradient descent updates the weights, step by step
4. Repeat until the error is minimized

Google Translate and ChatGPT run this same optimization loop at enormous scale.

Optimization: Descending the Error Surface



Feature Engineering

Mixing different features creates new features, through linear transformations and matrix operations. New, meaningful data emerges from existing data.

Example

Height + Weight → **BMI (Body Mass Index)**
a brand-new feature built from existing data.

LINEAR TRANSFORMATIONS (3D → 2D, 2D → 3D)

Image resizing, rotating, blurring, and zooming are all matrix multiplications — producing new information from existing data.

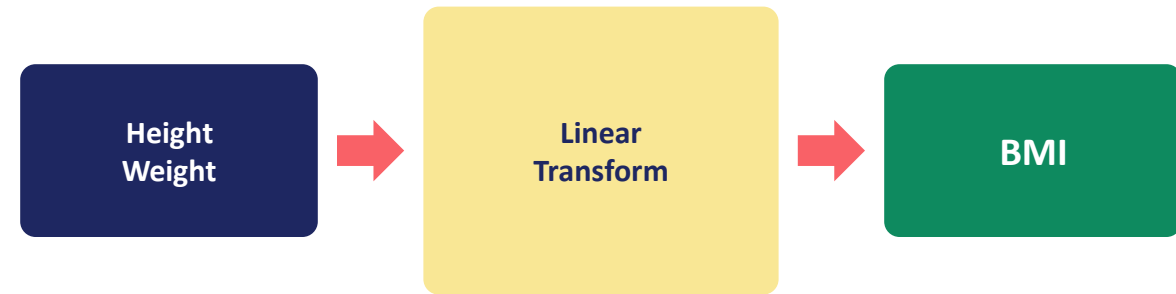


IMAGE OPERATIONS AS MATRIX MULTIPLICATION

Rotate · Blur · Zoom · Resize

Every visual effect you apply to a photo is a matrix acting on a matrix.

Similarity Measures & Embeddings

Embeddings turn words or sentences into a list of numbers (a vector) whose properties represent the meaning of that thing. This is how recommendation systems and AI chatbots compare meaning.

Vector Arithmetic

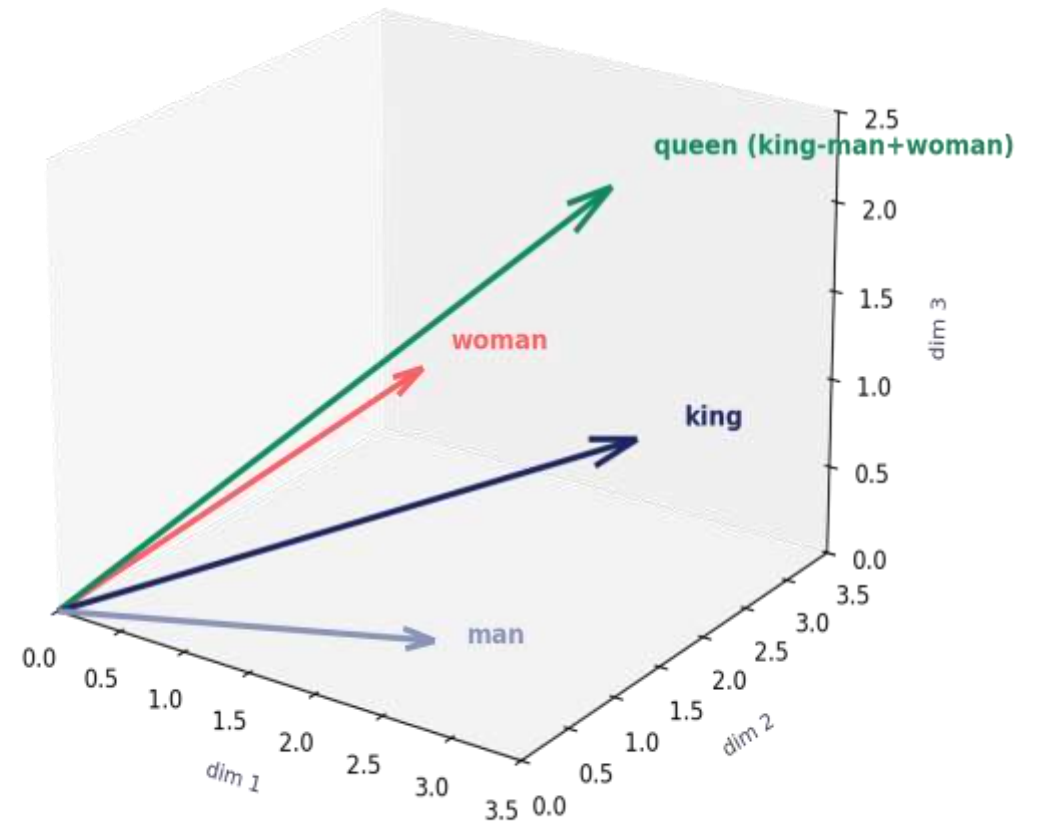
$$\text{king} - \text{man} + \text{woman} = \text{queen}$$

ChatGPT's word vectors encode relationships like this directly.

Spotify → compares song vectors to find similar songs

Similarity = dot product between two vectors

Vector Arithmetic in Word Embeddings



Remember Me

Linear Algebra = The Language of Machine Learning. Without it, there is no AI.

Data

= matrix

Training

= matrix operations

Compression

= matrix decomposition

Similarity

= dot product

Prediction

= matrix multiplication

THE AI PIPELINE, IN ONE LINE

Data in → find patterns → make predictions.

Every one of those steps is vectors and matrices at work. No linear algebra, no AI.

Scalar, Vector, and Matrix

Scalar

A single number

- No direction — just magnitude
- CS type: int, float, double
- e.g. `age = 20`
- C++: `float salary = 50000;`

Vector

A list of numbers

- Has both magnitude & direction
- CS type: array of floats
- e.g. `[2, -8, 7]` (Row 1)
- `float emp[2] = {5, 50000};`

Matrix

A table of numbers

- Organized in rows + columns
- e.g. `[-6 -4 27]`
- 3D data → `[3, 1, 2]` as a vector
- Vectors stack to form a matrix

Case Study: How Does an Email Spam Filter Work?

x

Spam word present?

Yes or No

y

“Lottery” word value

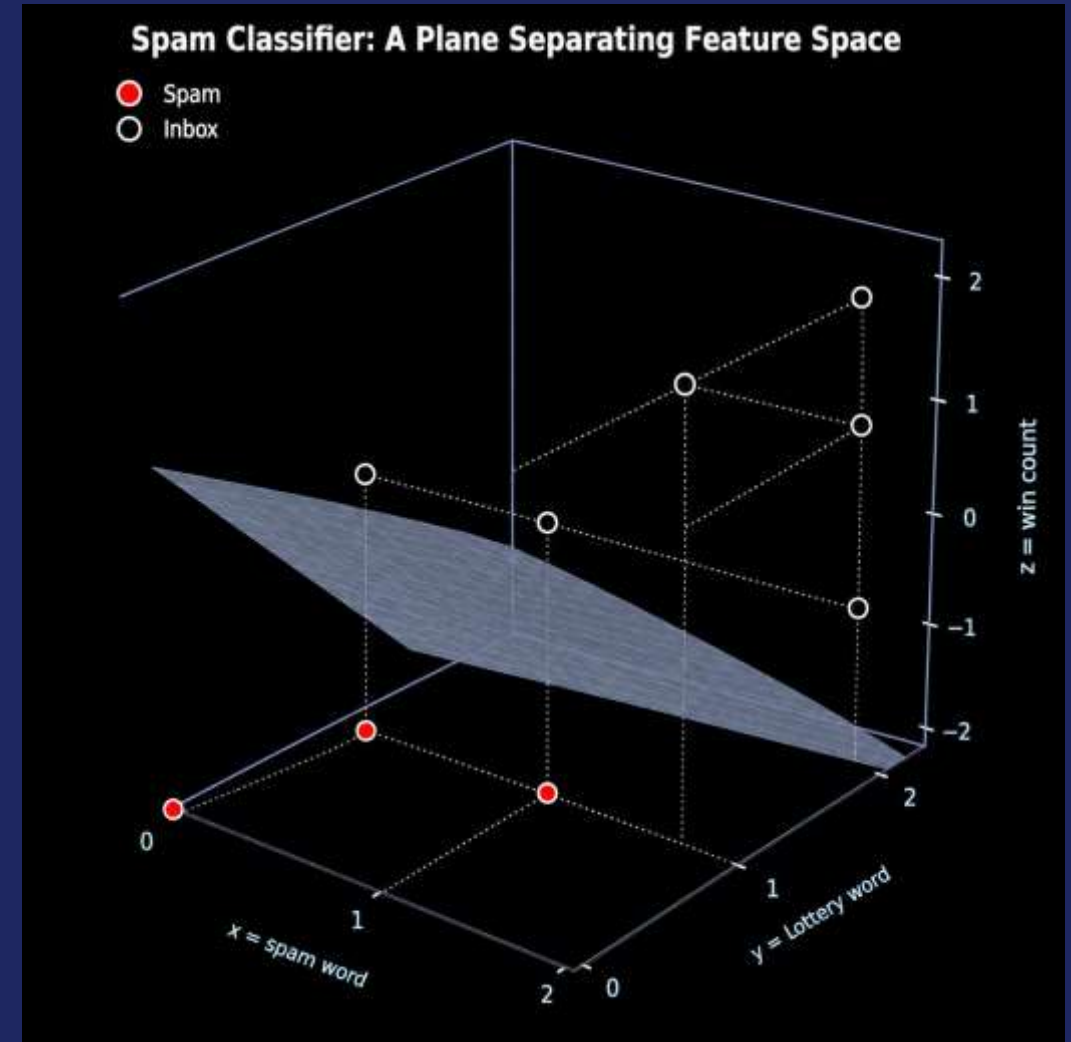
0, 1, or 2

z

“Win” word value

0, 1, or 2

Rule (classifier): Lottery + Win > Threshold



Spam	Lottery (x)	Win (y)
Yes	1	1
Yes	2	1
No	0	0
Yes	0	2
No	0	1
No	1	0
Yes	2	2
Yes	2	0
Yes	1	2

The Dot Product — How a Classifier Decides

Multiply each pair of corresponding numbers, then add the results.

$$A \cdot B = (a_1 \times b_1) + (a_2 \times b_2) + \dots$$

Worked example — Spam Model

Model (weight vector) $V = [1, 1]$ — both inputs matter equally.

Rule: if $\text{email} \cdot V > 1.5 \rightarrow \text{SPAM}$, otherwise $\rightarrow \text{INBOX}$

$$[1, 1] \cdot [1, 1] = 2$$

SPAM

$$[0, 0] \cdot [1, 1] = 0$$

INBOX

$$[2, 2] \cdot [1, 1] = 4$$

SPAM

$$[0, 1] \cdot [1, 1] = 1$$

INBOX

Same idea, different model

The weight vector V is fully customizable — change it and the classifier's behavior changes:

$[1, 0] \rightarrow$ only x matters

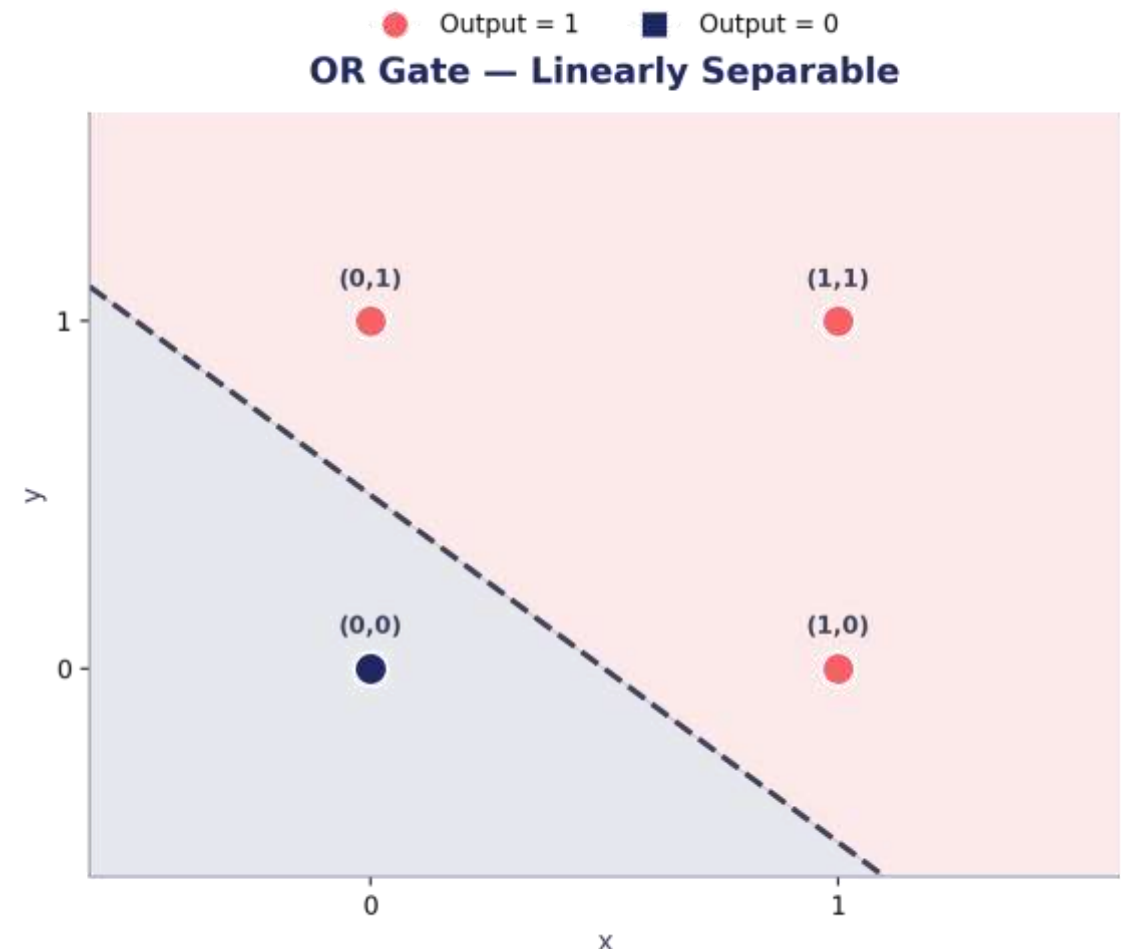
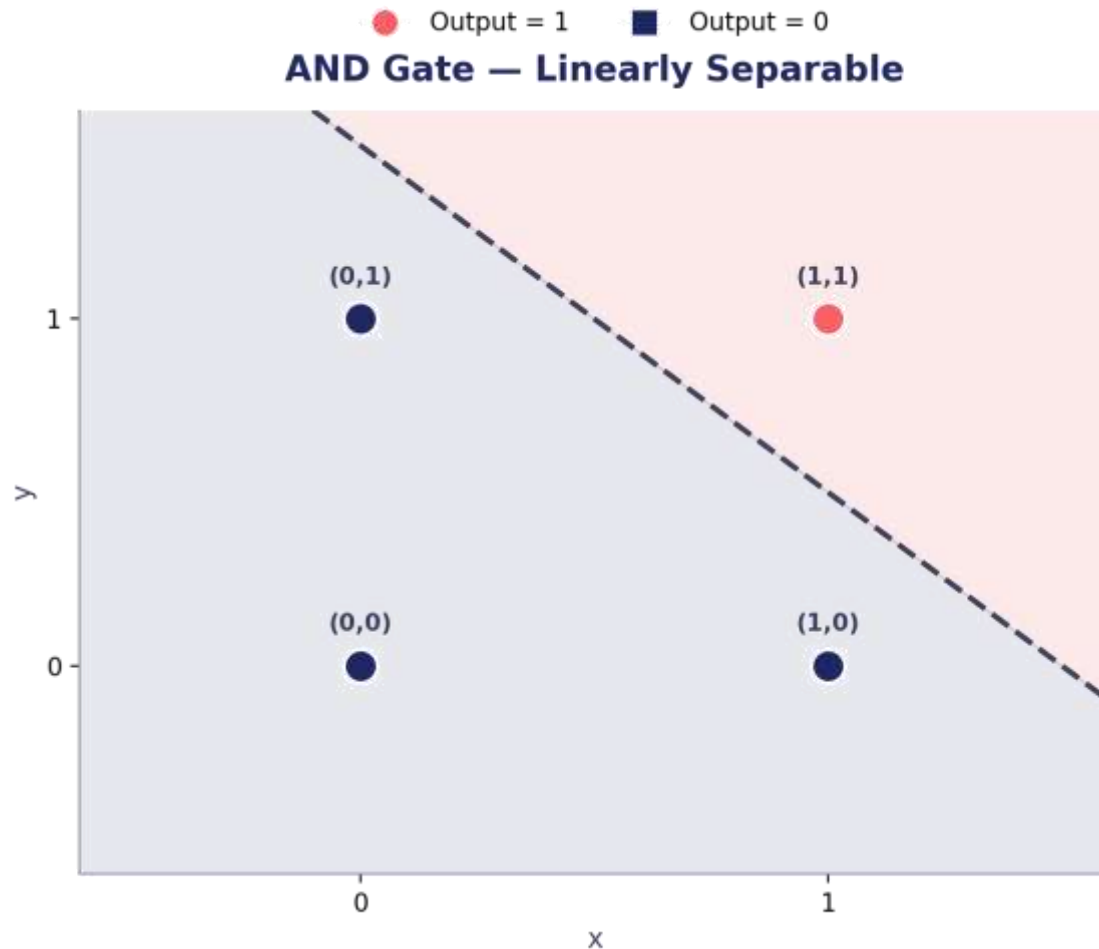
$[0, 1] \rightarrow$ only y matters

$[0, 0] \rightarrow$ no input matters

Any spam filter you can build, an inbox filter can be built the same way.

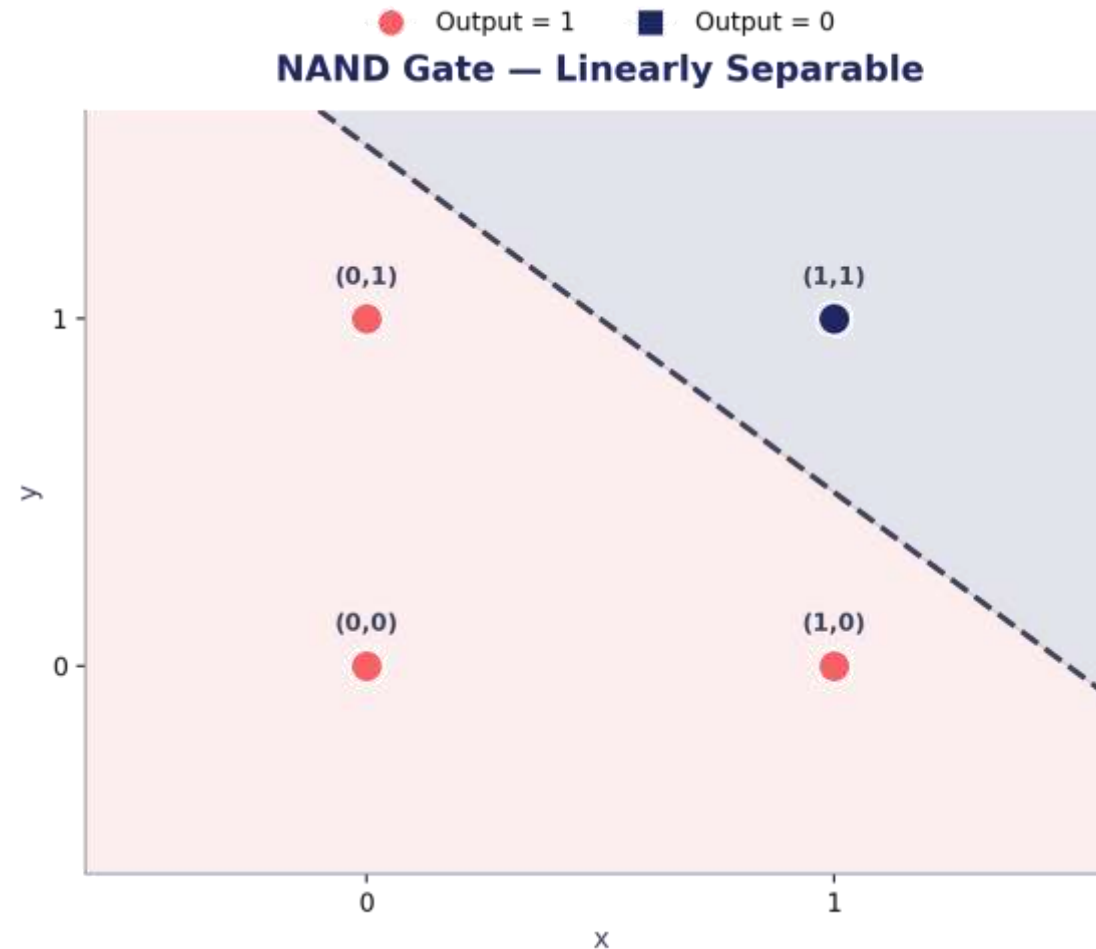
Logic Gates Are Linear Classifiers & have one layer neural network

Every truth table can be checked: can a straight line (a hyperplane) separate the outputs? If yes $\rightarrow$ one layer is enough. If no $\rightarrow$ we need more.



Flipping the Rule: NAND, NOR, and NOT

Changing the weight vector or the threshold reshapes the decision boundary — the same one-layer model can implement many different gates.



The XOR Problem

XOR is NOT linearly separable — a single-layer classifier cannot solve it, no matter how the weight vector is chosen.

XOR TRUTH TABLE

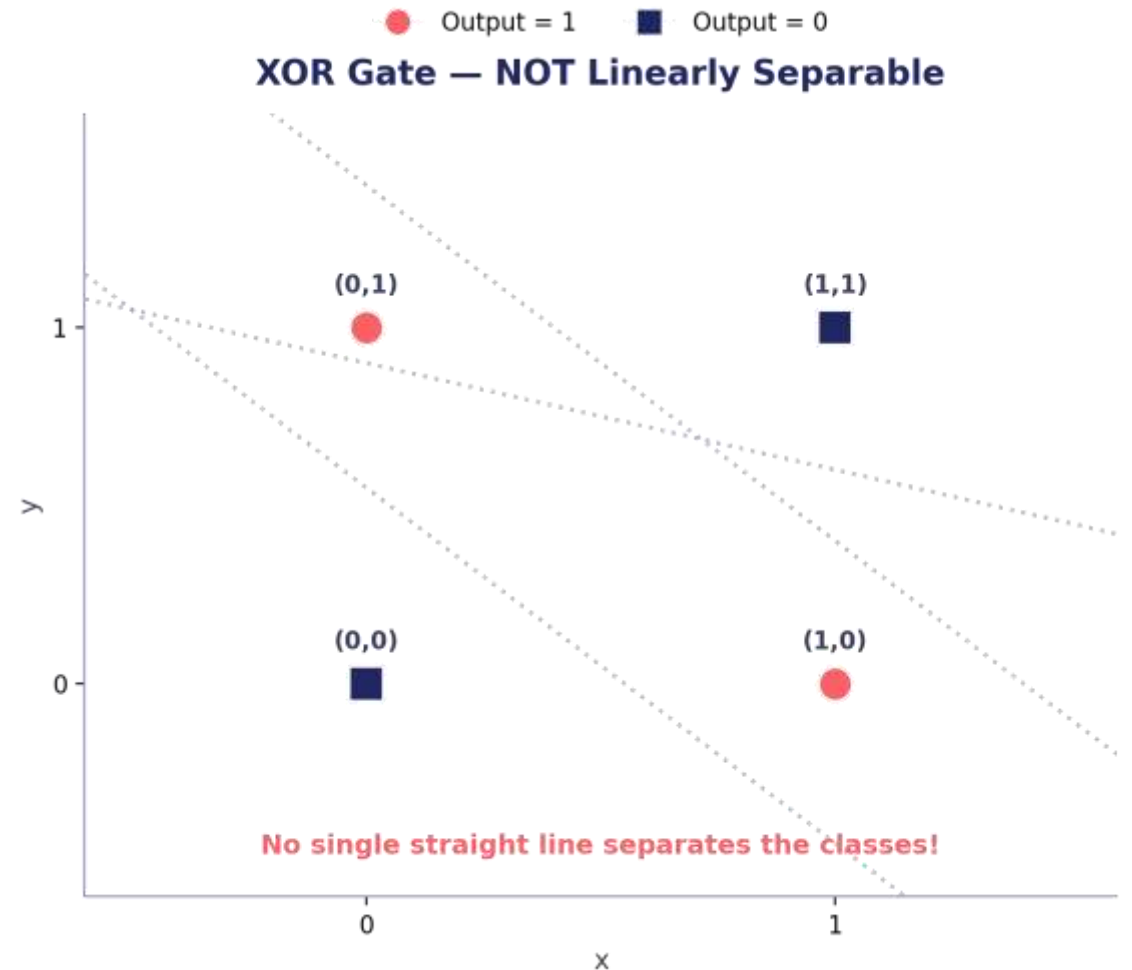
x	y	Output
0	0	0 (No)
0	1	1 (Yes)
1	0	1 (Yes)
1	1	0 (No)

Condition-based rule instead:

If $x = y \rightarrow \text{Output} = 0$

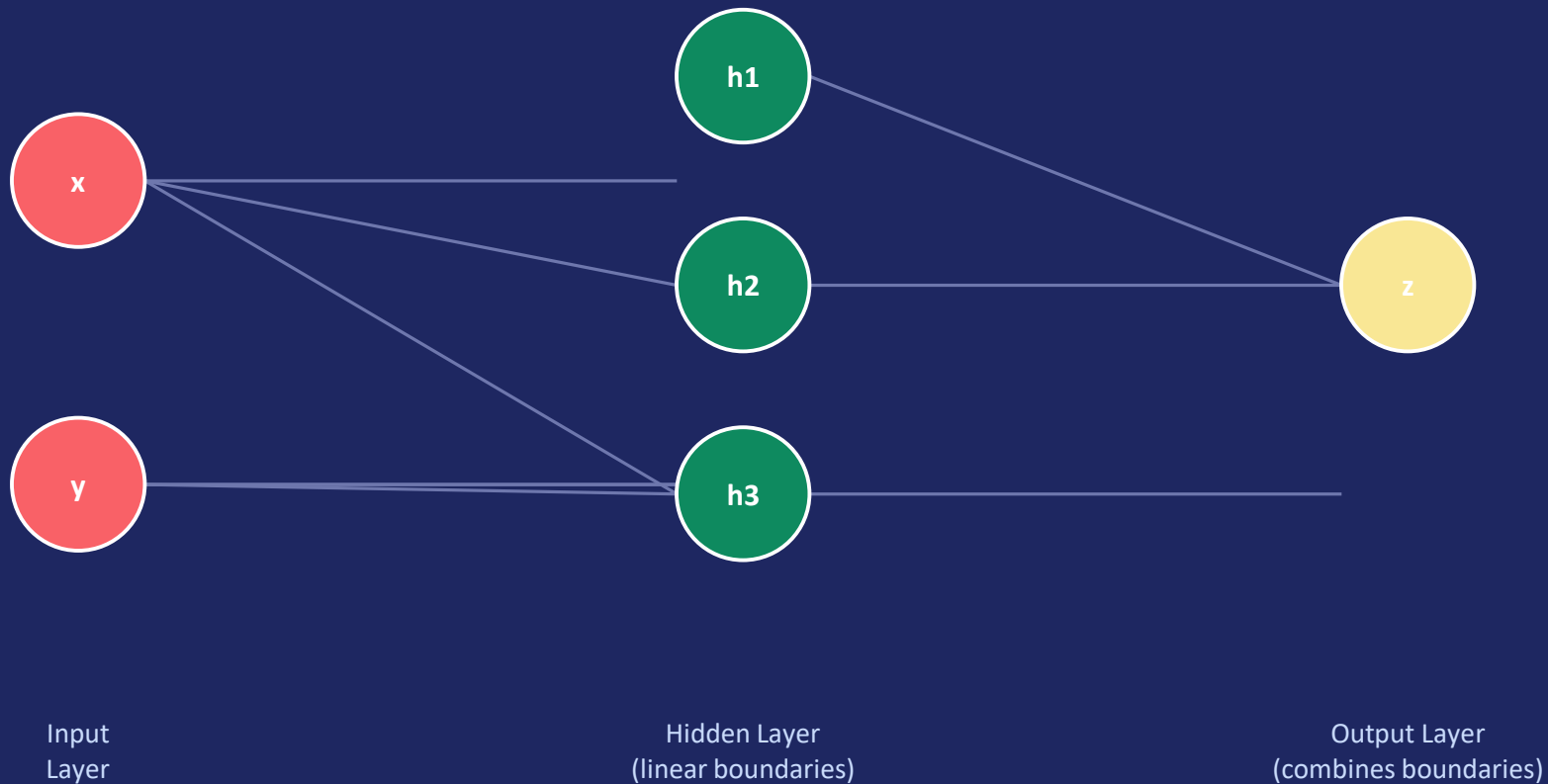
If $x \neq y \rightarrow \text{Output} = 1$

This condition can't be written as one straight boundary — we need to combine multiple lines.



Solving XOR: Stack the Layers

A single line can't separate XOR — but two lines, combined, can. This is exactly what a hidden layer in a neural network does.



Key idea

Hidden layer:
draws linear boundaries

Output layer:
combines those boundaries

Threshold:
acts as the activation step

*Result: a perfect mapping that
matches the XOR truth table.*